BigMart Item Price Prediction

CSE281 Introduction to AI - Project Report

Ali Nasser Sabry - 24P0248

May 2026

1. Problem Statement

This project solves the Kaggle competition `cse-281-spring-26-item-price-prediction`, a tabular regression problem based on the BigMart Sales dataset with anonymized columns `X1` through `X11`.

The objective is to predict the target column `Y` for 2523 unseen test rows using 6000 labeled training rows. The target is already log-transformed, so all models train and submit directly on the provided `Y` scale. No inverse transformation or additional `log1p` is applied.

The competition metric is Mean Absolute Error (MAE) on `Y`:

```
MAE = average(|actual Y - predicted Y|)
```

Lower scores are better. The best confirmed public leaderboard score from the final cleaned pipeline is `0.370`.

2. Dataset Overview

The input files are:

File	Rows	Purpose
`train.csv`	6000	Labeled training data with `Y`
`test.csv`	2523	Unlabeled rows for prediction
`sample_submission.csv`	2523	Required Kaggle submission format

The submission format is:

```
row_id,Y
0,<prediction for first test row>
1,<prediction for second test row>
...
```

`row_id` is a zero-based index matching the original order of `test.csv`.

Target distribution. `Y` is already log-transformed and lies mostly between 3.5 and 9.4.

Figure 1: Target distribution. `Y` is already log-transformed and lies mostly between 3.5 and 9.4.

Missingness summary. `X2` and `X9` are the main columns with missing values.

Figure 2: Missingness summary. `X2` and `X9` are the main columns with missing values.

2.1 Column Meaning

The anonymized columns correspond to the standard BigMart feature groups:

Column	Meaning	Handling
`X1`	item identifier	high-cardinality item ID; prefix and target encoding are used
`X2`	item weight	missing values imputed by item mean
`X3`	item fat content	dirty labels normalized
`X4`	item visibility	zeros treated as missing
`X5`	item type	categorical
`X6`	item MRP / price-like feature	strong numeric predictor
`X7`	outlet identifier	categorical and count feature
`X8`	outlet establishment year	converted to outlet age
`X9`	outlet size	missing values imputed by outlet type
`X10`	outlet location tier	categorical
`X11`	outlet type	categorical
`Y`	target	already log-transformed

3. Exploratory Data Analysis

The EDA confirmed the main data cleaning and modeling requirements.

4. Preprocessing And Feature Engineering

Feature engineering is implemented in `src/features.py` through the `BigMartFeatures` transformer. It is placed inside sklearn pipelines so each cross-validation fold fits preprocessing only on that fold's training rows.

4.1 Cleaning

The main cleaning steps are:

- Normalize `X3` from variants such as `LF`, `low fat`, and `reg` into clean categories.

Dirty values in `X3`. Multiple labels represent the same fat-content classes.

Figure 3: Dirty values in `X3`. Multiple labels represent the same fat-content classes.

X4 visibility has an abnormal spike at exactly zero, treated as missingness.

Figure 4: X4 visibility has an abnormal spike at exactly zero, treated as missingness.

X6 is a strong predictor of the target.

Figure 5: X6 is a strong predictor of the target.

- Override X3 to Non-Edible for item IDs whose prefix is NC.
- Treat X4 == 0 as missing visibility rather than a true zero.
- Impute X2 and X4 by item-level means with global fallback.
- Impute missing X9 by the mode within outlet type X11.

4.2 Engineered Features

The final engineered numeric features are:

Feature	Meaning
Outlet_Years	fixed reference year 2013 minus X8
X4_ratio	item visibility relative to outlet average visibility
X1_count	frequency of the item ID
X7_count	frequency of the outlet ID
X1_X7_count	frequency of the item-outlet pair
logX6	log-transformed price/MRP-like feature
X4_dev_X1	visibility deviation from the item mean

The final engineered categorical features are:

- X1_prefix
- X3
- X5
- X6_bin
- X7
- X9
- X10
- X11

The target-encoded features are:

- X1
- X5_X11

X5_X11 is an interaction between item type and outlet type.

The first two characters of X1 recover useful item meta-categories.

Figure 6: The first two characters of X1 recover useful item meta-categories.

Outlet type and outlet age show systematic target differences.

Figure 7: Outlet type and outlet age show systematic target differences.

Numeric correlation plot. `X6` is the most important raw numeric signal.

Figure 8: Numeric correlation plot. `X6` is the most important raw numeric signal.

4.3 Leakage Control

The highest-risk feature is `X1`, because it is a high-cardinality item ID. Target encoding is useful for this column, but it can leak target values if it is fit incorrectly.

Leakage is avoided as follows:

- All preprocessing is inside sklearn `Pipeline` objects.
- `TargetEncoder` is fit inside cross-validation.
- sklearn's `TargetEncoder` uses internal cross-fitting.
- Validation rows do not compute their encodings from their own target values.

The project also pools `test.csv` into target-independent count statistics such as `X1_count` and `X7_count`. This uses only feature columns and never reads hidden labels, so it is unsupervised transductive preprocessing, not target leakage.

5. Model Lineup

The final repository keeps exactly five active models in `src/models.py`:

Model	Family	Reason
`lasso_te200`	regularized linear model	strongest public-LB direction after calibration
`quantile_mae`	median / quantile regression	directly aligned with MAE
`rf`	random forest	bagged tree baseline
`lgbm`	LightGBM	gradient boosting baseline
`xgb`	XGBoost	second gradient boosting implementation

The neural network was removed from the active registry because it was slower, worse locally, and unnecessary under the five-model cap. A Huber model was also tested as a legacy probe, but it was not kept in the final active lineup.

5.1 Main Models

`lasso_te200` uses:

- `ElasticNet(alpha=0.003, l1_ratio=1.0)`
- `TargetEncoder(smooth=200.0)`

Because `l1_ratio=1.0`, this is effectively a Lasso model. It is well suited to the target-encoded item features because it regularizes noisy item effects.

LightGBM feature importance.

Figure 9: LightGBM feature importance.

`quantile_mae` uses:

- `QuantileRegressor(quantile=0.5, alpha=0.001)`

This model predicts the median, which is theoretically aligned with MAE. It achieved the best local out-of-fold MAE, although the calibrated Lasso performed best on the public leaderboard.

6. Validation Strategy

All models use the same shared 5-fold cross-validation setup:

```
KFold(n_splits=5, shuffle=True, random_state=42)
```

For each model, the code saves out-of-fold (OOF) predictions. An OOF prediction for a row is generated by a model that did not train on that row, making it a fair local validation estimate.

The final comparison table reports both:

- OOF MAE: the competition-aligned validation metric.
- OOF RMSE: an additional diagnostic that penalizes large errors more strongly.

7. Model Comparison

From `results/model_comparison.csv`:

model	oof_mae	oof_rmse	fit_seconds
quantile_mae	0.39627	0.52504	8.9
lasso_te200	0.40152	0.52041	0.7
xgb	0.40593	0.52442	125.7
lgbm	0.40813	0.52709	554.6
rf	0.41429	0.53351	54.4

The key observation is that `quantile_mae` achieved the best local OOF MAE, but the calibrated Lasso submission achieved the best public leaderboard result. This is not a contradiction: validation measures one split structure, while the public leaderboard measures a specific hidden public subset.

8. Feature Importance And Residuals

LightGBM feature importance is used as a model-agnostic sanity check for the engineered features.

The strongest raw signal is `X6`, the MRP/price-like feature. Outlet type and item-related features are also important, matching the EDA.

The residual plot below uses out-of-fold predictions from the best local model.

The residuals are roughly centered, with the largest errors appearing in the tails of the target distribution.

Predicted vs actual and residual histogram.

Figure 10: Predicted vs actual and residual histogram.

9. Main Diagnostic Insight

The most important project insight is that the test set has extremely high item overlap with the training set.

```
test["X1"].isin(train["X1"]).mean()  # approximately 0.993
```

Only 17 of 2523 test rows contain unseen item IDs. Therefore, the main task is not learning a very complex new representation; it is estimating item-level target behavior with appropriate regularization, then adjusting for outlet and price features.

This explains three observed facts:

1. Regularized linear models perform very well.
2. Many different models make highly similar predictions.
3. Public leaderboard scores are better than local CV scores because full training uses more item history than each CV fold.

10. Leaderboard Calibration

The best raw model was `lasso_te200`, with public score `0.378`. Since the public test distribution was easier than random CV and highly item-overlapped, a small affine calibration was tested:

```
prediction = mean(test_pred) + scale * (test_pred - mean(test_pred)) + shift
```

The final chosen calibration was:

```
scale = 1.05
shift = 0.08
```

This produced the best confirmed public score:

`0.370`

Final artifact:

`submissions/cand_lasso_scale1p05_shift0p08/submission.csv`

Calibration should be interpreted carefully. It improved the public leaderboard score, but it is post-processing of the best Lasso prediction vector, not a new model family. It may not transfer perfectly to a private split.

11. Final Results

Item	Result
Best local OOF MAE	`quantile_mae`, 0.39627
Best public leaderboard model	calibrated `lasso_te200`
Best public leaderboard score	0.370

Item	Result
Final submission artifact	`submissions/cand_lasso_scale1p05_shift0p08/submissio`
Active model count	5
Random seed	42

12. Reproducibility

The final cleaned repository is intended for LMS submission rather than further Kaggle uploads. The Kaggle deadline has passed, so `auto_submit.sh` and old probe artifacts were removed.

Environment:

- Python 3.12
- numpy
- pandas
- scikit-learn
- lightgbm
- xgboost
- matplotlib
- seaborn
- weasyprint / markdown for report rendering

Reproduce model comparison:

```
python -m src.train
```

Regenerate the final submission CSV locally:

```
python -m src.submit
```

Render this report:

```
python -m src.render_report
```

13. Conclusion

The final solution is intentionally compact. The central modeling decision is to exploit item identity safely through target encoding and regularized linear models. More complex models were useful for comparison, but they did not beat the calibrated Lasso on the public leaderboard.

The final active lineup satisfies the five-model constraint, the target encoding is leakage-controlled, the report includes EDA and diagnostics, and the final public score reached `0.370`.